HACKATHON ACTIVITY SCHEDULE

Postgraduate Assessment & Progress Management System

3-Day Competition · Three Groups · Four Students Per Group · Best Solution Wins

One complete system per group · Judged on 6 criteria

GROUP 1	GROUP 2	GROUP 3
Team name: _____	Team name: _____	Team name: _____
All 7 modules · Own architecture · Own UX · Compete for Ksh. 10,000	All 7 modules · Own architecture · Own UX · Compete for Ksh. 10,000	All 7 modules · Own architecture · Own UX · Compete for Ksh. 10,000

> **THE COMPETITION** *All three groups solve exactly the same problem and build the same complete system. What differs is how they build it. Judges compare three finished products at the end of Day 3.*

Section 1: How the Competition Works

1.1 Every Group Builds the Complete System

Each group of four students builds the entire Postgraduate Assessment & Progress Management System independently. This means all seven modules, all five user roles, a working database, a live deployed frontend, and a working demonstration. No features are divided between groups — every group is fully self-sufficient and fully competing.

1.2 Internal Group Structure — Four Roles

With four students per group, the most effective way to work is to assign each person a primary responsibility. These roles are not silos — everyone reviews everyone else's code — but they define who leads each area and who is accountable for each module's completion.

ROLE 1	ROLE 2	ROLE 3	ROLE 4
Team Lead + Full-Stack	Backend Developer	Frontend Developer	AI & DevOps Engineer
– Owns the overall architecture and ensures all parts connect – Writes the database schema and migration files – Resolves blockers when teammates are stuck	– Owns all API routes, controllers, and middleware – Implements JWT authentication and role-based access – Builds the business logic for Modules B, D, and E	– Owns all UI components, pages, and user flows – Implements role-based routing and protected pages – Builds dashboards for all 5 user roles	– Owns Module C — the AI transcript-to-corrections pipeline – Writes the AI prompt and parses the API response – Implements all notification logic (Module F)

– Implements the student pipeline state machine (Module A logic) – Does final integration testing and leads the demo	– Writes the seed data script – Manages the .env file and database connection	– Connects all frontend forms to backend API endpoints – Leads the UI/UX decisions and interface quality	– Manages GitHub — branches, PRs, merges, and tags – Leads deployment to the hosting platform on Day 3

> **COLLABORATION RULE** *Roles define accountability, not exclusivity. If the Frontend Developer finishes their sprint task early, they help the Backend Developer. If the AI Engineer is blocked, the Team Lead steps in. The team wins or loses together.*

1.3 The Seven Modules — Every Group Must Build All of Them

#	Module	What It Does	Primary Role	Sprint
A	**Pipeline Tracker**	Student stage dashboard + admin overview of all students	**Lead + Frontend**	Day 2 Sprint 1
B	**Presentation Booking**	Students request slots; admins confirm; seminar calendar for all	**Backend + Frontend**	Day 2 Sprint 2
C	**AI Corrections**	Transcript upload → AI extraction → chair validation → student checklist	**AI Engineer**	Day 2 Sprint 2
D	**Assessment Rubric**	Weighted scoring form for panel members; auto pass/fail calculation	**Backend + Frontend**	Day 2 Sprint 1
E	**Quarterly Reports**	Digital form + configurable multi-signature approval chain	**Backend + Frontend**	Day 2 Sprint 2
F	**Notifications**	In-app + email alerts for all key events and overdue warnings	**AI Engineer**	Day 2 Sprint 3
G	**Admin Dashboard**	PG Dean overview: all students, stages, days elapsed, upcoming events	**Frontend**	Day 2 Sprint 3

> **MVP PRIORITY** *Modules A, B, D, and E are the Minimum Viable Product. Complete these first. Modules C, F, and G are important differentiators — add them once the MVP is stable. A working 4-module system beats a crashing 7-module attempt.*

Section 2: Pre-Hackathon Setup

Students — Before Arriving

- Read the full Design Brief at least once. Arrive knowing which modules you find most interesting.
- Ensure your laptop has: Node.js v18+, Git, VS Code or Cursor IDE, and a database viewer (TablePlus or DBeaver).
- If you are unfamiliar with Git and GitHub, read Appendix B of this schedule — the step-by-step Git guide — before Day 1.

DAY 1 · UNDERSTAND · PLAN · SET UP

Reading the brief · Assigning roles · Designing the schema · First commit

Day 1 Philosophy

No feature code before 13:00. This is a hard rule. Groups that ignore it and start coding immediately always produce disconnected systems with broken data models they cannot fix by Day 3. The morning produces three planning outputs — a feature priority list, user stories converted to GitHub Issues, and a drawn database schema. These take less than three hours and save two days of confusion.

08:00	OPENING — All Groups Together (60 minutes)
08:00–08:15	Welcome, housekeeping, and competition rules. Facilitator presents: the prize, judging criteria, GitHub requirements, deployment requirements, and the 3-day timeline. Groups receive their database credentials and AI API keys.
08:15–08:50	Silent individual reading of the Design Brief. Every student annotates three things: features they understand, terms needing clarification, and one thing they would design differently. No group discussion yet — individual reading ensures every team member forms their own understanding.
08:50–09:10	Whole-room Q&A. Facilitator takes questions from any student and answers for the whole room. All clarifications are written on the shared board and stay visible for the rest of the hackathon.
09:10–09:30	Groups move to their tables and introduce themselves. Roles 1–4 are assigned. The GitHub owner (Role 4) forks the template repository, renames it with the group number, and invites all members. Confirm all three repos are created before proceeding.

> ⬆ **GIT PUSH [REPO SETUP]** Branch: **main** — Fork template, rename repo, add all members. Update README: group number, team name, member names and roles. Commit: 'chore: initial repo setup'

09:30	WITHIN-GROUP PLANNING (90 minutes)
09:30–10:00	Feature prioritisation. Each group classifies every module as Must Build (MVP), Should Build, or Could Build. Write this on paper. Only Must Build items are scheduled for Days 1 and 2 — stretch goals are for Day 3 morning if time allows.
10:00–10:30	User story writing. Write at least 8 user stories across all 5 roles using the format: 'As a [Student/Supervisor/Chair/Admin/PG Dean], I want to [action] so that [outcome].' These become GitHub Issues. Role 2 and Role 3 write the backend and frontend stories respectively.
10:30–11:00	Database schema design on paper or whiteboard. Every table, every key field, every relationship drawn before any code is written. Role 1 leads this. Agree on exact table names and field names — changing these later costs hours.

> **ROLE 4 TASK DURING PLANNING** *While the team plans on paper, the AI & DevOps Engineer sets up the GitHub Project board with columns (Backlog / In Progress / Review / Done). Create one Issue for each user story written by the team. Label issues: module letter (A–G) and day (day-1, day-2, day-3).*

11:00	**BREAK (15 minutes)**

11:15	**TECH STACK DECISION & GITHUB ISSUES (45 minutes)**
11:15–11:40	Whole-group technology decision. Agree on: frontend framework (React recommended), backend language and framework, database access method (Prisma ORM or raw SQL), UI component library if any, and deployment platform for Day 3. Role 1 records all decisions in the README under a 'Tech Stack' section. This is the last moment to change your mind.
11:40–12:00	GitHub Issues finalised. Role 4 confirms all issues are created with the correct labels. Assign day-1 issues to specific team members. Every team member should have at least one issue assigned and labelled day-1 before lunch.

> ⬆ **GIT PUSH [PLANNING COMPLETE]** Branch: **main** — README updated with tech stack decisions. All GitHub Issues created, labelled, assigned. Commit: 'chore: planning done — issues created, stack confirmed'

12:00	**LUNCH (60 minutes)**

13:00	**ENVIRONMENT SETUP & PROJECT SCAFFOLD (60 minutes)**
13:00–13:20	Facilitator demonstration for the whole room: initialise a Node.js project, connect to PostgreSQL, run a first migration, confirm the connection. Students follow along on their own machines.
13:20–14:00	Each group scaffolds their project. Role 1 and Role 2 set up the backend (project init, folder structure, database connection confirmed). Role 3 sets up the frontend (project init, routing, dev server confirmed). Role 4 commits the .env.example file and verifies .gitignore is blocking the real .env. All four machines must be running without errors before moving on.

> ⬆ **GIT PUSH [SCAFFOLD]** Branch: **feature/day1-setup** — Backend + frontend running, DB connection confirmed, folder structure in place, .env.example committed. Commit: 'chore: scaffold complete'

14:00	**DATABASE MIGRATION, SEEDS & AUTHENTICATION (3 hours)**
14:00–15:30	Role 1 and Role 2 write and run all database migration files. Every table from this morning's schema must be created. Role 3 installs frontend dependencies and builds the login screen while Roles 1 and 2 work on the database. Role 4 writes the seed script simultaneously — aim for realistic data: named students, real research topics, varied pipeline stages.
15:30–16:00	Seeds run and verified. Role 2 confirms all tables contain correct seed data using a database viewer. Role 1 reviews the schema against this morning's design — fix any field that was missed.
16:00–17:00	Authentication. Role 2 builds: register endpoint, login endpoint returning a JWT, and protected route middleware. Role 1 reviews the middleware logic. Role 3 builds the login page and connects it to the API. Role 4 tests all five roles (Student, Supervisor, Chair, Admin, Dean) in Postman, confirming each role receives a different dashboard view after login.

> ⬆ **GIT PUSH [END OF DAY 1]** Branch: **PR → main** — DB migrated, seeds running, auth working for all 5 roles. Screenshot of database tables in PR description. Merge before 17:30. Tag: v0.1-day1 | Commit: 'feat: schema, seeds, and authentication'

17:00	**END-OF-DAY STANDUP — All Groups (20 minutes)**
Format	Each group: 3 minutes. State: what is working, what is your first task tomorrow, any blocker that needs overnight mentor help.

Facilitator	Confirm all three PRs are merged. Any group without a working database and auth by end of Day 1 receives priority mentor support at 08:00 tomorrow.

DAY 2 · BUILD · BUILD · BUILD

Three focused sprints · All seven modules · End-to-end flow by 17:00

Day 2 Philosophy

Day 2 is the engine of the competition. Three sprints cover all seven modules. Work in parallel within your group — all four roles should have something in progress at every moment. The sprint structure below specifies who leads each piece, but all team members contribute. The integration smoke test at the end of Day 2 is the most important test of the hackathon — run it honestly and fix what it reveals.

08:00	MORNING KICKOFF (20 minutes)
08:00–08:10	Each group reviews the GitHub Issues board. Confirm which issues are targeted today. Every team member picks up exactly one In Progress issue.
08:10–08:20	Identify the internal dependency: which feature must be done first because others depend on it? For most groups this is the role-based routing layer — all dashboards depend on knowing who is logged in. Make this the first task.

Sprint 1 — 08:20 to 10:30 · Modules A & D

Sprint 1 builds the two most central features: the student pipeline tracker and the assessment rubric. These are built first because they represent the core value of the system and anchor the demo.

Time & Module	Sprint Goal & Tasks — All Three Groups Build This
08:20–09:30 **Module A** *Lead + Frontend*	**Student Pipeline Dashboard & Admin Student Overview** Every user logs in and sees a role-appropriate dashboard. Students see their stage; admins see all students. – Role 1 + Role 3: Build the student dashboard — current pipeline stage displayed by name, days elapsed at this stage, description of the next required action, a visual progress bar through all 8 stages. – Role 2: Build the backend state machine — GET /students/:id/stage returns stage name, gate conditions, and whether a transition is currently allowed. – Role 3: Build the admin student overview — a table of all students with name, programme, current stage, supervisor, days at stage, and a colour-coded status badge (green/amber/red). – Role 2: Implement role-based route guards — a 403 response when a student navigates to an admin-only page.
09:30–10:30 **Module D** *Backend + Frontend*	**Assessment Rubric Form & Score Engine** Panel members can score a presentation; the system calculates a pass/fail automatically. – Role 2: Build the rubric scoring endpoint — POST /presentations/:id/score accepts a panel member's weighted responses, stores them as JSONB, calculates a percentage, and determines pass/fail against a configurable threshold. – Role 3: Build the rubric form UI — at least three categories (Literature Review, Methodology, Presentation Quality) with Likert scale questions (1–5) and a submit button. – Role 2: Build the score aggregation — if multiple panel members score the same presentation, average their percentages into one final score. – Role 1: Connect the pass/fail outcome to the pipeline state machine — a passing score unlocks the 'advance stage' action on the student's dashboard.

> ⬆ **GIT PUSH [SPRINT 1]** Branch: **feature/sprint1** — Module A and Module D working with real DB data. Backend routes and frontend views both pushed. Commit: 'feat: sprint 1 — pipeline dashboard and assessment rubric'

10:30	**BREAK (15 minutes)**

Sprint 2 — 10:45 to 13:00 · Modules B, C & E

Sprint 2 is the most demanding sprint. Three complex modules run in parallel. Role 4 owns Module C entirely. Roles 2 and 3 split Modules B and E. Role 1 floats between all three, unblocking where needed.

Time & Module	Sprint Goal & Tasks — All Three Groups Build This
10:45–11:30 **Module B** *Backend + Frontend*	**Presentation Booking & Seminar Calendar** Students request slots; admins confirm; a calendar shows all upcoming presentations. – Role 2: Build booking endpoints — POST /presentations/request (student submits preferred date/time); PATCH /presentations/:id/confirm (admin approves); GET /presentations/calendar (all scheduled presentations). – Role 3: Build the booking request form for students and the admin confirmation queue — a list of pending requests with confirm/reschedule buttons. – Role 3: Build the seminar calendar view — visible to all department members; shows student name, stage, date, time, venue.
10:45–12:30 **Module C** *AI Engineer (Role 4)*	**AI Transcript → Corrections Pipeline** Upload a transcript, extract structured corrections using AI, validate as chair, deliver to student. – Build the transcript upload interface — a text area where the chair pastes the raw meeting transcript. – Write the AI API call — send the transcript to Claude/OpenAI with a prompt that asks for a JSON array: [{description, category, urgency: critical|major|minor}]. Test this with a sample transcript before building the UI. – Build the chair validation screen — AI output displayed as a list; chair can approve, edit text, change urgency, or delete each item. Nothing reaches the student until the chair clicks 'Send to Student'. – Build the student corrections checklist — each correction shown with urgency badge; student marks items addressed; submit for supervisor sign-off when all done.
11:00–13:00 **Module E** *Backend + Frontend + Lead*	**Quarterly Report Form & Multi-Signature Approval Chain** Students fill a digital progress report that routes automatically through a configurable chain of approvers. – Role 2: Build the approval chain engine — configurable sequence stored in the database. Masters: Supervisor 1 → Supervisor 2 → Dean → Finance. PhD: adds a third supervisor. After each approval, system automatically notifies the next approver. – Role 2: Build report endpoints — POST /reports (student submits); PATCH /reports/:id/approve (approver acts); GET /reports/:id/status (approval chain progress). – Role 3: Build the report form — sections for progress summary, objectives achieved, challenges, next quarter plan, and supervisor comments. – Role 1: Build the approval chain status view — a visual chain showing each approver as a node: tick (approved), clock (pending), X (returned). Visible to student, supervisors, and admin.

> ⬆ **GIT PUSH [SPRINT 2]** Branch: **feature/sprint2** — Modules B, C, E pushed. AI API call tested with a real transcript sample. Approval chain routing working. Commit: 'feat: sprint 2 — booking, AI corrections, quarterly reports'

13:00	**LUNCH (45 minutes)**

Sprint 3 — 13:45 to 16:00 · Modules F, G & Integration

Sprint 3 completes the system and then connects it. After building Modules F and G, every group runs one complete student journey end to end. This is the most important test of Day 2.

Time & Module	Sprint Goal & Tasks — All Three Groups Build This
13:45–14:30 **Modules F & G** *AI Engineer + Frontend*	**Notifications & Administrator Dashboard** Complete the remaining two modules so all seven are implemented. – Role 4 — Module F: implement in-app notification bell (unread count badge in header) and email notifications triggered by: presentation booked, corrections sent to student, quarterly report submitted, report approved or returned, stage advanced. – Role 3 — Module G: build the PG Dean / admin overview — student count per pipeline stage, students overdue at their stage (30+ days amber, 60+ days red), upcoming presentations in the next 14 days, and pending approval actions requiring the Dean's attention.
14:30–16:00 **All Modules** *Whole Team*	**End-to-End Integration Test — The Most Important Hour of Day 2** Run one complete student journey through the entire system without any manual database edits. – Run this journey: Log in as Student → view pipeline stage and next action → submit presentation booking request → log in as Admin → confirm the booking → log in as Panel Member → submit rubric score → confirm pass/fail triggers correctly → log in as Chair → upload a transcript → validate AI-extracted corrections → log in as Student → view corrections checklist and mark items addressed → fill quarterly report → log in as Supervisor → approve the report → confirm chain routes to next approver → verify student's pipeline stage advances on their dashboard. – Fix every broken step before ending this session. Prioritise flow over completeness — a smooth 6-module journey beats a broken 7-module attempt. – Load realistic demo data — give the demo student a real name, a meaningful research topic, and a plausible pipeline stage. This is the data judges will see.

> ⬆ **GIT PUSH [END OF DAY 2]** Branch: **PR → main** — All modules attempted. End-to-end journey tested. PR description: list what works, what is partial, what you fix tomorrow. Merge before 16:30. Tag: v0.2-day2 | Commit: 'feat: day 2 — all modules, integration tested'

16:30	END-OF-DAY STANDUP (20 minutes)
Format	3 minutes per group. State honestly: which modules work end-to-end, what is your first fix tomorrow, will you be ready to deploy by 11:00.
Facilitator	Any group without Modules A, D, and E working in some form by end of Day 2 receives priority mentor support at 08:00 on Day 3.

DAY 3 · FIX · DEPLOY · WIN

Morning fixes · Feature freeze 11:00 · Deployment · Demo rehearsal · Final judging

Day 3 Philosophy

The groups that win on Day 3 are not the ones who build the most new features in the morning — they are the ones who stabilise what they have, deploy confidently, and present clearly. Feature freeze is at 11:00 and is absolute. After that, only bug fixes and deployment configuration are permitted on main.

> **FEATURE FREEZE AT 11:00 — ABSOLUTE** *No new feature code merges to main after 11:00. Only bug fixes, demo data corrections, and deployment config. This rule protects every group from breaking a working system in the final hours. Role 4 enforces this on behalf of the group.*

08:00	MORNING FIX SESSION (90 minutes)
08:00–08:15	Each group reviews yesterday's PR description — specifically the 'what is broken' list. Assign every fix to one person. No unassigned bugs.
08:15–09:15	Fix sprint. Work only on bugs from Day 2. No new features. Role 4 manages the fix branches — each fix gets its own branch (fix/description) and its own PR.
09:15–09:30	Smoke test. Re-run the full student journey from Day 2. Count steps that work without intervention. If all core steps work, move to stretch goals. If steps are still broken, continue fixing until 10:00.
09:30–10:45	Stretch goals — only if core flow is complete. Recommended: AI correction urgency colour badges; configurable pass threshold in admin settings; PDF export of a fully signed quarterly report; 'days at stage' visual warning on the admin dashboard.

> ⬆ **GIT PUSH [BUG FIXES]** Branch: **fix/[issue-name]** — One small PR per fix. Descriptive messages: 'fix: approval chain skipping second supervisor'. Merge all before 10:45.

11:00	FEATURE FREEZE & DEPLOYMENT (60 minutes)
11:00	Role 4 calls feature freeze. All open branches merged. Tag the commit: git tag v1.0-demo and push the tag.
11:00–11:45	Role 4 leads deployment. Recommended: backend on Railway or Render, frontend on Vercel, database already on Supabase. Role 1 sets environment variables in the hosting dashboard. Roles 2 and 3 test the deployed URL — confirm login works with real credentials and real data appears.
11:45–12:00	Role 3 updates the README with: live deployment URL, working features list, local setup instructions. Role 4 commits this and verifies the final README on GitHub.

> ⬆ **GIT PUSH [DEPLOYMENT]** Branch: **main** — README updated with live URL and setup guide. .env.example complete. Commit: 'chore: deployed to [URL] — README updated'

12:00	LUNCH + DEMO PREPARATION (90 minutes)
12:00–12:30	Lunch.
12:30–13:30	Demo preparation. Rehearse the 8-minute demo at least twice. Assign demo roles: Role 3 drives the laptop (they know the UI best), Role 1 narrates, Roles 2 and 4 handle judge questions. Use the demo script below.
13:30–13:50	Submit to facilitator via the group chat: (1) GitHub repository URL, (2) live deployment URL, (3) reflection document uploaded to /docs in the repo. All three required before presentations begin.

Demo Script — 8 Minutes

0:00–1:00	1. THE PROBLEM (1 minute)
What to say	Make the problem concrete. Name a real pain point: 'Students at this university wait up to three months between their seminar and receiving corrections, because the minutes are handwritten and depend on one person typing them up.' Then state: 'We built a system that solves this.'
1:00–5:30	**2. THE LIVE DEMO (4.5 minutes)**
What to show	Navigate the deployed URL. Show one complete student journey. Do not apologise for missing features — only show what works. The judges will navigate the app themselves after your demo.
5:30–7:00	**3. ONE TECHNICAL HIGHLIGHT (90 seconds)**
What to say	Pick your most technically impressive decision. Explain it in plain language, not code. Example: 'We built the approval chain as a configurable sequence in the database, so the administrator can change the signing order without touching the code.'
7:00–8:00	**4. Q&A (1 minute)**
What to expect	Judges will probe missing features or unusual choices. Answer honestly. 'We ran out of time for X but here is how we would build it' is a strong answer.

14:00	FINAL PRESENTATIONS & JUDGING (2 hours 30 minutes)
14:00–14:20	Group 1 — 8 minutes presentation + 12 minutes judge Q&A
14:20–14:40	Group 2 — 8 minutes presentation + 12 minutes judge Q&A
14:40–15:00	Group 3 — 8 minutes presentation + 12 minutes judge Q&A
15:00–15:40	Judges deliberate — 40 minutes. Judges review the three GitHub repos independently while deliberating.
15:40–16:30	Results, prize award (Ksh. 10,000 to winning group), and closing retrospective. Facilitator leads: what would the best-of-three combined system look like? What would you build next?

One-Page Group Reflection (Submit Before 13:50)

Upload as a document to /docs in your repo. Max 100 words per answer.

- What is the most technically interesting thing your group built and how does it work in plain language?
- What was the hardest problem you encountered and how did you solve or work around it?
- Which feature are you most proud of and why does it matter to the actual users of this system?
- What would you build next if you had two more days?
- Looking at the problem again — is there anything the Design Brief got wrong, or a better approach you discovered while building?

> ⬆ **GIT PUSH [FINAL SUBMISSION]** Branch: **main** — Reflection in /docs, clean README, all issues closed. Tag: v1.0-final | Commit: 'docs: final submission'

Appendix A: Judging Rubric

All three groups are scored on the same criteria. The group with the highest total score wins Ksh. 10,000. In a tie, the group with the higher Functionality score wins.

Criterion	0	1–2	3–4	5	Weight & Notes
Functionality & Completeness	Nothing works	1–2 modules	3–5 modules	All 7 end-to-end	30% — Tested on live URL. Judges navigate the app.
Technical Quality	No structure	Works but messy	Reasonable structure	Clean, readable, documented	20% — Judges review the GitHub repo: structure, naming, DB relationships, no hardcoded secrets.
User Experience & Design	Unusable	Confusing	Clear and navigable	Intuitive for non-technical staff	20% — Could a university administrator use this on their first try?
Innovation & Problem-Fit	Misses problem	Addresses some issues	Good with gaps	Genuinely solves the PG problem	15% — Does this system address accountability, assessment, and workflow failures?
Presentation & Demo	Unclear or crashes	Partial structure	Clear, mostly rehearsed	Compelling, confident	10% — Structure, clarity, honest handling of limitations.
Deployment & Docs	Not deployed	Localhost only	Deployed, basic README	Live URL + full README + reflection	5% — Live URL, setup guide, .env.example, reflection in /docs.

Appendix B: Git & GitHub Step-by-Step Guide

This guide covers everything you need for the hackathon. It assumes you are using VS Code or Cursor as your IDE. All commands are typed into the Terminal inside VS Code (Menu → Terminal → New Terminal) or Cursor's built-in terminal.

> **WHAT IS GIT?** *Git is a version control tool that runs on your computer. GitHub is the website where your code is stored online. Git tracks every change you make. GitHub shares those changes with your team and keeps a backup of everything.*

Step 1 — One-Time Setup (Do This First, Before Anything Else)

Tell Git who you are. This name and email appear on every commit you make.

```
# Set your name and email — use your real name so your teammates know who made each change
git config --global user.name "Your Full Name"
git config --global user.email "your.email@university.ac.ke"
```

Verify it worked:

```
git config --global user.name
```

> **VS CODE TIP** *In VS Code, press Ctrl+` (backtick) to open the integrated terminal. In Cursor, use the same shortcut or go to View → Terminal. You should see a prompt like: your-name@computer:~/your-folder$*

Step 2 — Fork and Clone the Repository (Day 1, 09:10)

The GitHub owner (Role 4) does the fork on GitHub.com. Then every team member clones it to their own laptop.

Role 4 Only — Fork on GitHub.com

- Go to the template repository URL given by your facilitator
- Click the Fork button (top right on GitHub)
- Name the repo: pg-system-group-1 (replace 1 with your group number)
- Go to Settings → Collaborators → Add your three teammates by their GitHub username

All Four Members — Clone to Your Laptop

Copy the repository URL from GitHub (green Code button → HTTPS). Then in your terminal:

```
# Replace the URL with your group's actual repository URL
git clone https://github.com/your-username/pg-system-group-1.git
cd pg-system-group-1
```

Open the folder in VS Code:

```
code .
```

> **WHAT JUST HAPPENED?** *git clone downloaded a full copy of the repository to your laptop. The cd command moved your terminal into that folder. Now every git command you run applies to this project.*

Step 3 — The Daily Git Workflow (Use This Every Time You Write Code)

This is the sequence you will repeat dozens of times over three days. Learn it on Day 1 and it becomes automatic.

3a. Before Starting Work — Always Pull First

Before you write a single line of code each session, get the latest changes from your teammates:

```
# Always do this before starting work to avoid conflicts
git pull origin main
```

> **WHY THIS MATTERS** *If you skip the pull and your teammate has pushed changes, your code and their code will conflict. Always pull first — it takes two seconds and prevents an hour of headaches.*

3b. Create a Feature Branch

Never work directly on main. Create a new branch for each piece of work:

```
# Create a new branch and switch to it
git checkout -b feature/student-dashboard
```

Branch naming convention for this hackathon:

- `feature/day1-setup` · `sprint1` · `sprint2` · `sprint3` · `student-dashboard` · `auth`
- `fix/approval-chain-routing` · `login-error` · `db-connection`
- `chore/gitignore` · `env-example` · `readme-update`

3c. Write Code — Then Stage, Commit, and Push

After writing code that does something meaningful (not after every single line), save your progress:

```
# Step 1: See what files have changed
git status
```

```
# Step 2: Stage the files you want to commit
# To stage everything changed:
git add .
# To stage one specific file:
git add src/routes/auth.js
```

```
# Step 3: Commit with a meaningful message
```

```
git commit -m "feat: student pipeline dashboard showing current stage"
```

```
# Step 4: Push your branch to GitHub
git push origin feature/student-dashboard
```

> **COMMIT MESSAGE FORMAT** *Use these prefixes so your teammates know what each commit does: feat: for new features · fix: for bug corrections · chore: for setup and config · docs: for README and documentation changes. Example: 'fix: quarterly report approval chain skipping second supervisor'*

Step 4 — Opening a Pull Request (PR)

When your branch is ready to be reviewed and merged into main, open a Pull Request on GitHub.com:

1. Go to your repository on GitHub.com
2. You will see a yellow banner: 'feature/student-dashboard had recent pushes — Compare & pull request'. Click it.
3. Write a title and description. In the description: what does this PR do? What should reviewers test? Are there any known issues?
4. Assign at least one teammate as a reviewer
5. Click Create Pull Request

Your teammate reviews it, and when happy, clicks Merge Pull Request. Then everyone pulls the updated main:

```
git checkout main
git pull origin main
```

Step 5 — Handling Merge Conflicts

A merge conflict happens when two people changed the same line in the same file. Git does not know which version to keep, so it asks you to decide. This is normal — don't panic.

When you pull and see a conflict, open the file in VS Code. You will see markers like this:

```
<<<<<<< HEAD
const port = 3000;  // your version
=======
const port = 5000;  // teammate's version
>>>>>>> feature/backend-setup
```

VS Code shows conflict markers with helpful buttons above them: Accept Current Change, Accept Incoming Change, or Accept Both. Choose the correct version, delete the marker lines, save the file, then:

```
git add  .
git commit -m "fix: resolve merge conflict on port configuration"
git push origin your-branch-name
```

> **PREVENTION IS BETTER THAN CURE** *The best way to avoid conflicts is to pull before starting work, keep branches short-lived, and communicate with your team about which files you are each editing. If you and a teammate both need to edit the same file, coordinate — don't both edit it at the same time.*

Step 6 — Tagging a Release

At the end of each day and at feature freeze, create a tag. Tags are permanent markers in your history — they let you jump back to any working state.

```
# Create a tag
git tag v0.1-day1
# Push the tag to GitHub
git push origin v0.1-day1
```

Tags required for this hackathon:

- `v0.1-day1` — end of Day 1, after auth and DB are working
- `v0.2-day2` — end of Day 2, after all modules are pushed
- `v1.0-demo` — feature freeze at 11:00 on Day 3
- `v1.0-final` — final submission at 13:50 on Day 3

Step 7 — Critical .gitignore Rules

The .gitignore file tells Git which files to never track or push. These must be in your .gitignore from Day 1 and must never be removed:

```
# Environment variables — NEVER push these
.env
# Node.js dependencies — these are installed locally, not stored in Git
node_modules/
# Python virtual environment
venv/
__pycache__/
# Build outputs
dist/
build/
```

Always commit a .env.example file with placeholder values so teammates know which environment variables are needed:

```
# .env.example — commit this, it shows what variables are needed without real values
DATABASE_URL=postgresql://username:password@host:port/database
JWT_SECRET=your-secret-key-here
AI_API_KEY=your-anthropic-or-openai-key-here
PORT=3000
```

Step 8 — Quick Command Reference Card

Keep this page open in a browser tab or print it. These are the commands you will use most.

What You Want to Do	Command	When to Use It
Get latest changes from GitHub	**git pull origin main**	Before starting work each session

What You Want to Do	Command	When to Use It
See what files have changed	`git status`	Before staging — always check first
See detailed changes in a file	`git diff filename.js`	To review what changed before committing
Create and switch to a new branch	`git checkout -b feature/my-feature`	When starting a new piece of work
Switch to an existing branch	`git checkout main`	When moving between branches
Stage all changed files	`git add .`	After writing code worth saving
Stage one specific file	`git add src/routes/auth.js`	When only some changes are ready
Commit staged changes	`git commit -m "feat: describe what you built"`	After staging — write a clear message
Push branch to GitHub	`git push origin feature/my-feature`	After committing — share with the team
Push main after a merge	`git push origin main`	After merging a PR and pulling main
See branch history	`git log --oneline`	To see recent commits
See all branches	`git branch -a`	To see local and remote branches
Create a version tag	`git tag v1.0-demo`	At feature freeze and final submission
Push a tag to GitHub	`git push origin v1.0-demo`	After creating the tag locally
Undo last commit (keep changes)	`git reset --soft HEAD~1`	If you committed too early — recovers your changes
Discard all unsaved changes	`git checkout -- .`	CAREFUL — permanently removes uncommitted work

> **IF YOU ARE EVER STUCK** *Ask your Role 4 teammate first — they are the GitHub owner and should know the most Git commands. If the whole group is stuck, ask the facilitator. Never try to force-push or delete branches without asking for help — this can permanently remove your teammates' work.*

Appendix C: Complete GitHub Push Schedule

All three groups follow this identical push schedule. The content of each push differs between groups — the timing does not.

Day	Time	Branch	Commit Tag	Must Include
Day 1	**09:30**	**main**	initial-setup	Repo forked, README with group name and all 4 member names, .gitignore committed
Day 1	**11:00**	**main**	planning-done	README updated with tech stack, all GitHub Issues created, labelled, and assigned to members
Day 1	**17:00**	**PR→main**	day1-done	Database migrated, seeds running, auth working for all 5 roles — tag: v0.1-day1
Day 2	**10:30**	**feature/sprint1**	sprint1	Module A pipeline dashboard + Module D rubric — backend and frontend both working
Day 2	**13:00**	**feature/sprint2**	sprint2	Modules B, C, E — booking, AI corrections, quarterly report form and approval chain
Day 2	**16:00**	**PR→main**	day2-done	All modules integrated, end-to-end flow tested — tag: v0.2-day2
Day 3	**09:30**	**fix/[name]**	fixes	All bug fixes — one small PR per fix, descriptive commit messages
Day 3	**11:00**	**main**	freeze	FEATURE FREEZE — final state, all branches merged — tag: v1.0-demo
Day 3	**11:45**	**main**	deploy	README updated with live URL, .env.example complete
Day 3	**13:45**	**main**	final	Reflection doc in /docs, clean README, all issues closed — tag: v1.0-final

End of Hackathon Activity Schedule

Three groups. One problem. One prize. Best solution wins.